DSA Practical No: 08

Name: Tushar Balaso Solankar PRN: B26ET2021 Div: SY3 Class: C3

Binary Search Tree

INPUT:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *left;
   struct node *right;
};

struct node* create(struct node *root, int value)
{
   if (root == NULL)
   {
      struct node *newnode;
      newnode = (struct node*)malloc(sizeof(struct node));

      newnode->data = value;
      newnode->left = NULL;
      newnode->right = NULL;

      return newnode;
```

```
    }

    if (value < root->data)
        root->left = create(root->left, value);
    else if (value > root->data)
        root->right = create(root->right, value);

    return root;
}

void search(struct node *root, int key)
{
    if (root == NULL)
    {
        printf("Element not found\n");
        return;
    }

    if (root->data == key)
        printf("Element found\n");
    else if (key < root->data)
        search(root->left, key);
    else
        search(root->right, key);
}

void inorder(struct node *root)
{
```

```
    if (root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct node *root)
{
    if (root != NULL)
    {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct node *root)
{
    if (root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

```
int main()
{
   struct node *root = NULL;
   int choice, value;

   while (1)
   {
      printf("\n\n1. Create/Insert");
      printf("\n2. Search");
      printf("\n3. Inorder Traversal");
      printf("\n4. Preorder Traversal");
      printf("\n5. Postorder Traversal");
      printf("\n6. Exit");
      printf("\nEnter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
         case 1:
            printf("Enter value: ");
            scanf("%d", &value);
            root = create(root, value);
            printf("Node inserted successfully\n");
            break;

         case 2:
            printf("Enter value to search: ");
            scanf("%d", &value);
```

```
            search(root, value);
            break;

        case 3:
            printf("Inorder: ");
            inorder(root);
            printf("\n");
            break;

        case 4:
            printf("Preorder: ");
            preorder(root);
            printf("\n");
            break;

        case 5:
            printf("Postorder: ");
            postorder(root);
            printf("\n");
            break;

        case 6:
            return 0;

        default:
            printf("Invalid choice\n");
    }
```

}

OUTPUT:

```
1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 1
Enter value: 20
Node inserted successfully


1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 1
Enter value: 30
Node inserted successfully


1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 1
Enter value: 40
Node inserted successfully
```

```
1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 1
Enter value: 50
Node inserted successfully


1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 1
Enter value: 70
Node inserted successfully


1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 3
Inorder: 20 30 40 50 70
```

```
1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 4
Preorder: 20 30 40 50 70


1. Create/Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 5
Postorder: 70 50 40 30 20
```